



El libro del curso

Python con Pokémon

Aprendé a programar desde cero, paso a paso, con Linux, Python y mucha aventura.

Y convertite en Campeón de Kanto

Manual + libro de Python · `generar_manual.py`

Índice

0. Cómo usar este libro y el curso

- 0.1. ¿Qué es este proyecto?
- 0.2. Instalación
- 0.3. La Liga Pokémon: jugá tu progreso
- 0.4. El método: cómo trabajar cada semana

1. Tu primer programa en Python

- 1.1. `print()`: mostrar cosas en pantalla
- 1.2. El REPL: probar al instante
- 1.3. Comentarios
- 1.4. Cuando algo sale mal: los errores

2. Variables y tipos de datos

- 2.1. Variables
- 2.2. Los cuatro tipos básicos
- 2.3. f-strings: armar texto con variables

3. Operadores

- 3.1. Operadores aritméticos
- 3.2. Operadores de comparación
- 3.3. Operadores lógicos: `and`, `or`, `not`
- 3.4. Operadores con texto

4. Cadenas de texto (strings)

- 4.1. Largo, búsqueda e índices
- 4.2. Métodos de strings
- 4.3. `split` y `join`
- 4.4. Slicing: rebanar texto

5. Pedir datos al usuario: `input()`

- 5.1. `input()` siempre devuelve texto
- 5.2. Conversión de tipos

6. Condicionales: `if` / `elif` / `else`

- 6.1. `if`: hacer algo solo si...
- 6.2. `else`: el 'si no'
- 6.3. `elif`: más caminos

6.4. Combinar condiciones

7. Bucles: repetir cosas (while y for)

- 7.1. while: repetir mientras...
- 7.2. for: recorrer una secuencia
- 7.3. break y continue
- 7.4. Bucles anidados

8. Funciones

- 8.1. Definir y llamar una función
- 8.2. Parámetros: pasarle datos
- 8.3. return: devolver un resultado
- 8.4. Valores por defecto
- 8.5. Scope: dónde vive cada variable
- 8.6. Funciones lambda
- 8.7. Recursión

9. Listas y tuplas

- 9.1. Acceder y modificar
- 9.2. Métodos de listas
- 9.3. enumerate y zip
- 9.4. Comprensiones de listas
- 9.5. Tuplas

10. Diccionarios y conjuntos

- 10.1. Diccionarios: pares clave → valor
- 10.2. Conjuntos (sets)
- 10.3. Resumen de colecciones

11. Archivos y manejo de errores

- 11.1. Abrir archivos con with
- 11.2. CSV: datos en filas y columnas
- 11.3. try / except: manejar errores

12. Programación orientada a objetos (POO)

- 12.1. Definir una clase
- 12.2. Métodos: las acciones del objeto
- 12.3. __str__: cómo se muestra el objeto
- 12.4. Encapsulamiento básico

13. POO avanzado: herencia y polimorfismo

- 13.1. Herencia
- 13.2. Polimorfismo
- 13.3. property, métodos de clase y abstractas

14. Módulos, pip y entornos virtuales

- 14.1. import y la librería estándar
- 14.2. json: guardar datos estructurados
- 14.3. Tus propios módulos
- 14.4. pip y entornos virtuales
- 14.5. requests: traer datos de internet

15. Linux: la terminal

- 15.1. Moverte por las carpetas
- 15.2. Crear, copiar y borrar
- 15.3. Rutas absolutas y relativas
- 15.4. Permisos básicos
- 15.5. Resumen

16. Linux: comandos intermedios

- 16.1. Editar con nano
- 16.2. Variables, redirección y pipes
- 16.3. Buscar: grep y find
- 16.4. Procesos y permisos
- 16.5. Tu primer script bash
- 16.6. Instalar programas y SSH
- 16.7. Resumen

17. Mapa de temas del curso

- 17.1. Fase 1 — Linux
- 17.2. Fase 2 — Python básico
- 17.3. Descanso — Git
- 17.4. Fase 3 — Python avanzado
- 17.5. Fase 4 — Proyectos

18. Tests, preguntas frecuentes y glosario

- 18.1. Cómo correr los tests
- 18.2. Preguntas frecuentes

18.3. Glosario

CAPÍTULO 0. Cómo usar este libro y el curso

¡Bienvenido! Este libro tiene **dos caras**. Por un lado es el **manual** del proyecto: te explica cómo instalarlo y cómo avanzar. Por otro lado es un **libro completo** que te enseña Linux y Python desde cero, con explicaciones paso a paso y muchos ejemplos.

Está pensado para alguien que **nunca programó**. No hace falta saber nada de antes. Vamos con calma, con temática Pokémon, y cada concepto se explica primero en palabras simples y después con código que podés copiar y probar.

0.1. ¿Qué es este proyecto?

Es un curso completo de **Linux** y **Python**, dividido en 12 semanas (más una semana de descanso de Git) y 4 proyectos finales. Cada concepto se explica con una analogía Pokémon, cada ejercicio usa Pokémon, y cada proyecto es algo que un Entrenador usaría de verdad.

Este libro acompaña al curso: podés leerlo de principio a fin como un libro, o usarlo como referencia para buscar un tema puntual cuando lo necesites.

0.2. Instalación

Abrí una terminal dentro de la carpeta del proyecto y corré:

```
BASH
bash setup.sh
```

Eso crea un entorno virtual (una cajita aislada para las librerías del curso) e instala lo necesario. Después, cada vez que vayas a trabajar, lo activás:

```
BASH
source venv/bin/activate
```

CUIDADO

Si `setup.sh` falla, abrílo con un editor: está comentado línea por línea. Las dos primeras semanas son de Linux y no necesitan Python para los ejercicios de terminal.

0.3. La Liga Pokémon: jugá tu progreso

La mejor forma de hacer el curso. En vez de "hacer la tarea", subís de nivel. Hay un único programa que es tu centro de mando:



```
BASH
python aventura.py
```

Desde ese único archivo podés:

- **Elegir un capítulo para jugar:** lanza el juego interactivo de esa semana. Si tenés progreso, **continuás donde lo dejaste**; si ya lo completaste, te ofrece **reintentarlo**.
- **Entrenar** una semana: corre tus ejercicios y te da EXP por cada test que pasás. Si algo falla, te dice **qué ejercicio** y una pista.
- **Combates de gimnasio:** con una medalla en mano, retás al líder con un desafío integrador más difícil.
- Ver tu **Tarjeta de Entrenador**, las **8 medallas**, los **logros** y el mapa.
- Mantener tu **racha** diaria.

Cuando consigas las 8 medallas (y venzas a los 8 líderes), ¡sos **Campeón de Kanto!**

0.4. El método: cómo trabajar cada semana

1. **Leé** la teoría de la semana (o el capítulo de este libro).
2. **Resolvé** los ejercicios (escribí tu código donde dice `# TU CÓDIGO ACÁ`).
3. **Probá** con los tests (ver capítulo 18).
4. **Jugá** el `interactivo.py` de la semana.
5. **Entrená** esa semana en la Liga para ganar EXP.

TIP

Regla de oro: equivocarse es parte del juego. Escribí el código vos mismo, no copies y pegues: tipear te enseña. Y cuando algo no funcione, leé el mensaje de error con calma: casi siempre te dice qué pasó.

CAPÍTULO 1. Tu primer programa en Python

Python es un lenguaje de programación: la forma en que le damos órdenes a la computadora. Un programa es una **lista de instrucciones** que la computadora ejecuta una por una, de arriba hacia abajo, en orden.

Python es famoso porque se lee casi como inglés y es ideal para aprender. Lo usan para webs, videojuegos, inteligencia artificial, análisis de datos y mucho más.

ANALOGÍA POKÉMON

Pensá en Python como el **idioma de los Entrenadores**. Vos escribís instrucciones (el código) y la computadora las obedece, como un Pokémon bien entrenado obedece a su dueño. Cuanto más claras las órdenes, mejor responde.

1.1. print(): mostrar cosas en pantalla

La instrucción `print()` muestra texto en la pantalla. Es lo primero que hace todo programador. Lo que va entre las comillas se llama **cadena de texto** (o string):

```
print("¡Hola, mundo Pokémon!")  
print("Elegí a Pikachu como inicial")
```

Cuando corrés ese programa, en la pantalla aparece:

```
¡Hola, mundo Pokémon!  
Elegí a Pikachu como inicial
```

Imprimir varias cosas juntas

Podés pasarle a `print()` varias cosas separadas por comas. Las muestra en la misma línea, separadas por un espacio:

```
print("Mi Pokémon es", "Charizard")    # Mi Pokémon es Charizard  
print("Nivel:", 50)                    # Nivel: 50
```


Saltos de línea y separadores

Dentro de un texto, `\n` es un **salto de línea** y `\t` es una **tabulación** (un espacio grande). También podés cambiar el separador con `sep` y el final con `end`:

```
print("Línea 1\nLínea 2")      # imprime dos líneas
print("Pika", "chu", sep="-")  # Pika-chu
print("Hola", end=" ")        # no salta de línea al final
print("mundo")                # ...así "Hola mundo" queda junto
```

1.2. El REPL: probar al instante

Si escribís `python3` en la terminal, se abre el **REPL** (modo interactivo): escribís una línea, apretás Enter, y ves el resultado al toque. Es ideal para experimentar sin crear un archivo. El símbolo `>>>` es su prompt (te indica que espera tu instrucción):

```
>>> 2 + 2
4
>>> "Pika" + "chu"
'Pikachu'
>>> print("Hola")
Hola
```

Para salir del REPL escribís `exit()` y Enter. Para programas de verdad (que se guardan y reusan), escribís el código en un archivo terminado en `.py` y lo corrés así:

```
python3 mi_programa.py
```

NOTA

En muchos sistemas el comando es `python3`; en otros, `python`. Si uno no funciona, probá el otro.

1.3. Comentarios

Un **comentario** es texto que Python **ignora** por completo. Sirve para explicar tu código (a otros y a tu yo del futuro). Empieza con el símbolo `#`:

```
# Esto es un comentario: Python no lo ejecuta.
print("Hola")      # también se puede comentar al final de una línea

# Los comentarios sirven para:
# - explicar qué hace una parte del código
# - dejar notas o recordatorios
# - "apagar" temporalmente una línea sin borrarla
```

TIP

Comentar bien tu código es de buena programadora. Pero ojo: no comentes lo obvio (`x = x + 1` # suma 1 a x no aporta). Comentá el **porqué**, no el qué.

1.4. Cuando algo sale mal: los errores

Equivocarse es normal y constante al programar. Si escribís algo que Python no entiende, te muestra un **error**. No te asustes: el error casi siempre te dice qué pasó y en qué línea. Por ejemplo, si te olvidás una comilla:

```
print("Hola)
# SyntaxError: unterminated string literal (detected at line 1)
```

Leé la última línea del error: dice el **tipo** de error (`SyntaxError`) y una pista. Con el tiempo, leer errores se vuelve natural.

TIP

Truco profesional: si no entendés un error, copió el mensaje exacto y pegalo en un buscador. El 99% de las veces, alguien ya lo tuvo y está explicado. Esto es la mitad de programar.

CAPÍTULO 2. Variables y tipos de datos

2.1. Variables

Una **variable** guarda un dato con un nombre, para poder usarlo después. Es como una **Pokéball**: adentro guardás algo y le ponés una etiqueta con un nombre. El símbolo `=` se llama **asignación**: "guardá lo de la derecha en el nombre de la izquierda".

```

nombre = "Pikachu"      # guardamos un texto en la variable 'nombre'
nivel = 25               # guardamos un número en la variable 'nivel'

print(nombre)           # Pikachu
print(nivel)            # 25

```

Las variables se pueden cambiar

Podés cambiar el valor de una variable cuando quieras: el nuevo valor reemplaza al viejo. Por eso se llaman "variables" (varían):

```

nivel = 25
print(nivel)      # 25
nivel = 26        # ahora vale otra cosa
print(nivel)      # 26

nivel = nivel + 1  # tomá el valor actual, sumale 1, y volvé a guardarlo
print(nivel)      # 27

```

Reglas para los nombres

Los nombres de variables tienen algunas reglas y costumbres:

- Solo letras, números y guión bajo `_`. **No** pueden empezar con número.
- No llevan espacios: usá guión bajo para separar palabras (`pokemon_inicial`).
- Distinguen mayúsculas: `nivel` y `Nivel` son variables distintas.
- Elegí nombres que se entiendan: `nivel` es mejor que `n`.

```


```

```
pokemon_inicial = "Charmander" # bien
hp_maximo = 100 # bien
# 3pokemon = "no" # mal: no puede empezar con número
# mi pokemon = "no" # mal: no puede tener espacios
```

2.2. Los cuatro tipos básicos

Cada dato en Python tiene un **tipo**. El tipo define qué se puede hacer con ese dato. Los cuatro tipos que más vas a usar al empezar son:

Tipo	Qué es	Ejemplos
<code>int</code>	Número entero (sin decimales)	<code>25</code> , <code>0</code> , <code>-7</code>
<code>float</code>	Número con decimales	<code>6.0</code> , <code>3.14</code> , <code>-0.5</code>
<code>str</code>	Texto, siempre entre comillas	<code>"Pikachu"</code> , <code>"a"</code> , <code>""</code>
<code>bool</code>	Verdadero o falso	<code>True</code> , <code>False</code>



PYTHON

```
nivel = 25 # int
peso = 6.0 # float (¡el punto lo hace decimal!)
nombre = "Pikachu" # str (entre comillas)
es_legendario = False # bool (con mayúscula: True o False)
```

Para ver el tipo de cualquier valor, usás la función `type()`:



PYTHON

```
print(type(25)) # <class 'int'>
print(type(6.0)) # <class 'float'>
print(type("Pikachu")) # <class 'str'>
print(type(True)) # <class 'bool'>
```

CUIDADO

Cuidado con un error común: "25" (con comillas) es **texto**, no un número. `25 + 5` da 30, pero `"25" + "5"` da "255" (pega los textos). El capítulo 5 explica cómo convertir entre tipos.

2.3. f-strings: armar texto con variables

Muchas veces querés mezclar texto con el valor de tus variables. La forma más cómoda y moderna es el **f-string**: ponés una `f` antes de las comillas y metés las variables entre `{llaves}`:

```
nombre = "Pikachu"
nivel = 25
print(f"Mi {nombre} es nivel {nivel}")    # Mi Pikachu es nivel 25
```

Adentro de las llaves podés poner hasta operaciones; Python las calcula:

```
hp = 100
print(f"{nombre} tiene {hp} HP")          # Pikachu tiene 100 HP
print(f"Al subir, será nivel {nivel + 1}") # Al subir, será nivel 26
```

Dar formato a los números

Para mostrar decimales con cierta cantidad de cifras, usás `:.2f` (dos decimales). Útil para porcentajes, precios, etc.:

```
porcentaje = 2 / 3
print(f"Victorias: {porcentaje:.2f}")    # Victorias: 0.67
```

CONCEPTO CLAVE

Los f-strings son la forma recomendada de armar texto. Antes se usaban otras (con `+` o con `.format()`), pero el f-string es más claro y corto. ¡Usalo siempre!

CAPÍTULO 3. Operadores

Los **operadores** son símbolos que combinan o comparan valores. Ya usaste uno: el `=` de asignación. Ahora vemos los de matemática, comparación y lógica.

3.1. Operadores aritméticos

Sirven para hacer cuentas:

Operador	Hace	Ejemplo	Resultado
<code>+</code>	Suma	<code>10 + 5</code>	15
<code>-</code>	Resta	<code>10 - 5</code>	5
<code>*</code>	Multiplicación	<code>10 * 5</code>	50
<code>/</code>	División (da decimal)	<code>10 / 4</code>	2.5
<code>//</code>	División entera	<code>10 // 4</code>	2
<code>%</code>	Resto (módulo)	<code>10 % 4</code>	2
<code>**</code>	Potencia	<code>2 ** 3</code>	8

/ contra //

La división normal `/` siempre da un `float` (decimal), aunque dé exacto. La división entera `//` descarta los decimales:

```

PYTHON
print(10 / 2)    # 5.0    (¡ojo, con punto: es float!)
print(10 / 3)    # 3.3333333333333335
print(10 // 3)   # 3      (entera: tira la parte decimal)
```

El módulo % es más útil de lo que parece

El **módulo** da el resto de una división. Sirve, por ejemplo, para saber si un número es par (resto 0 al dividir por 2):

```

PYTHON
```

```
print(10 % 3)    # 1    (10 = 3*3 + 1, sobra 1)
print(8 % 2)     # 0    (8 es par: no sobra nada)
print(7 % 2)     # 1    (7 es impar: sobra 1)
```

El orden de las operaciones

Python respeta el orden matemático: primero `**`, después `*`, `/`, `//`, `%`, y al final `+` y `-`. Usá paréntesis para forzar el orden que querés:

```
print(2 + 3 * 4)    # 14    (primero 3*4=12, después +2)
print((2 + 3) * 4)  # 20    (los paréntesis van primero)
```

3.2. Operadores de comparación

Comparan dos valores y devuelven un **bool** (`True` o `False`). Son la base de las decisiones (capítulo 6):

Operador	Significa	Ejemplo	Resultado
<code>=</code>	igual a	<code>25 = 25</code>	True
<code>≠</code>	distinto de	<code>25 ≠ 30</code>	True
<code>></code>	mayor que	<code>30 > 25</code>	True
<code><</code>	menor que	<code>30 < 25</code>	False
<code>≥</code>	mayor o igual	<code>25 ≥ 25</code>	True
<code>≤</code>	menor o igual	<code>20 ≤ 25</code>	True

CUIDADO

El error más común de principiante: confundir `=` con `==`. Un solo `=` asigna (guarda en una variable); el doble `==` compara. `nivel = 25` guarda; `nivel == 25` pregunta "¿son iguales?".

3.3. Operadores lógicos: and, or, not

Combinan condiciones. `and` es verdadero si **ambas** son verdaderas; `or` si **al menos una** lo es; `not` **invierte** (lo verdadero lo hace falso y viceversa).

Expresión	Resultado
<code>True and True</code>	True
<code>True and False</code>	False
<code>True or False</code>	True
<code>False or False</code>	False
<code>not True</code>	False

```

nivel = 30
tipo = "Fuego"

# Evolucionar si tiene nivel suficiente Y es de Fuego:
print(nivel ≥ 25 and tipo == "Fuego")  # True

# Es fuerte si es legendario O tiene nivel 100:
es_legendario = False
print(es_legendario or nivel == 100)    # False

# Puede pelear si NO está debilitado:
debilitado = False
print(not debilitado)                    # True

```

3.4. Operadores con texto

El `+` también funciona con strings: los **pega** (concatena). Y el `*` repite un texto:

```

print("Pika" + "chu")  # Pikachu
print("na" * 4)        # nananana
print("=" * 20)        # ===== (útil para separadores)

```


CAPÍTULO 4. Cadenas de texto (strings)

Un **string** es texto, siempre entre comillas (simples `'...'` o dobles `"..."`, da igual). Es uno de los tipos que más vas a usar: nombres, mensajes, datos de archivos, todo es texto.

```
nombre = "Pikachu"
frase = 'Yo elijo a Pikachu'
vacío = "" # un string vacío (sin nada adentro)
```

4.1. Largo, búsqueda e índices

Con `len()` sabés cuántos caracteres tiene. Con `in` preguntás si un texto está adentro de otro. Y con corchetes `[]` accedés a un carácter por su **posición** (que empieza en 0):

```
nombre = "Pikachu"
print(len(nombre)) # 7 (tiene 7 letras)
print("ka" in nombre) # True (¿"ka" está adentro?)
print(nombre[0]) # P (el primer carácter, posición 0)
print(nombre[1]) # i
print(nombre[-1]) # u (el último, contando desde atrás)
```

CONCEPTO CLAVE

Los **índices empiezan en 0**, no en 1. Entonces, en "Pikachu", la P está en la posición 0, la i en la 1, etc. Y `[-1]` es el último carácter. Esto vale también para listas (capítulo 9).

4.2. Métodos de strings

Un **método** es una acción que un dato sabe hacer. Se llama con un punto: `texto.metodo()`. Los strings tienen muchos. Importante: los métodos **no cambian** el original, devuelven uno **nuevo**:

```
nombre = "Pikachu"
print(nombre.upper()) # "PIKACHU" (mayúsculas)
print(nombre.lower()) # "pikachu" (minúsculas)
```

```
print(nombre.strip())      # "Pikachu"      (saca espacios de los bordes)
print("pikachu".capitalize()) # "Pikachu"  (primera letra en mayúscula)
print("Pikachu".replace("a", "@")) # "Pik@chu" (reemplaza)
print("Pikachu".startswith("Pika")) # True
print("Pikachu".endswith("chu"))   # True
print("Pikachu".count("a"))        # 1      (cuántas veces aparece "a")
```

Método	Qué hace
<code>.upper()</code> / <code>.lower()</code>	A mayúsculas / minúsculas
<code>.strip()</code>	Saca espacios de los bordes
<code>.capitalize()</code>	Primera letra en mayúscula
<code>.replace(a, b)</code>	Cambia todas las apariciones de a por b
<code>.startswith(x)</code> / <code>.endswith(x)</code>	¿Empieza / termina con x?
<code>.count(x)</code>	Cuántas veces aparece x
<code>.split(sep)</code> / <code>sep.join(lista)</code>	Separar / unir (abajo)

4.3. split y join

`split` **parte** un texto en una lista usando un separador. `join` hace lo contrario: **une** una lista en un texto. Son clave para trabajar con archivos CSV (capítulo 11):

```
linea = "Pikachu,Electrico,25"
partes = linea.split(",")
print(partes)      # ['Pikachu', 'Electrico', '25']

datos = ["Charizard", "Fuego", "50"]
linea = ",".join(datos)
print(linea)       # "Charizard,Fuego,50"
```

4.4. Slicing: rebanar texto

El **slicing** saca una "rebanada" del texto con `[inicio:fin]`. Toma desde `inicio` hasta `fin`, pero **sin** incluir el `fin`:



PYTHON

```
texto = "Pikachu"
#      0123456      (posiciones)

print(texto[0:4])    # "Pika"      (del 0 al 3)
print(texto[4:])     # "chu"       (del 4 hasta el final)
print(texto[:4])     # "Pika"      (del principio al 3)
print(texto[-3:])    # "chu"       (los últimos 3)
print(texto[::-1])   # "uhcakiP"   (todo, pero al revés)
```

CUIDADO

Los strings son inmutables: no podés cambiar una letra con `texto[0] = "X"` (da error). Si querés un texto distinto, creás uno nuevo (por ejemplo con `.replace()`).

CAPÍTULO 5. Pedir datos al usuario: input()

Hasta ahora tus programas mostraban cosas. Con `input()` tu programa puede **pedirle datos a quien lo usa**. Cuando Python llega a un `input()`, **se detiene** y espera que la persona escriba algo y apriete Enter. Lo que escribió queda guardado:

```
nombre = input("¿Cómo se llama tu Pokémon? ")
print(f"¡Hola, {nombre}!")
```

Si la persona escribe `Pikachu` y aprieta Enter, el programa muestra `¡Hola, Pikachu!`. El texto que le pasás a `input()` es la **pregunta** que se ve en pantalla.

5.1. input() siempre devuelve texto

¡CUIDADO!

¡MUY IMPORTANTE! `input()` SIEMPRE devuelve un `str` (texto), aunque la persona escriba un número. Por eso, si querés hacer cuentas, primero tenés que **convertirlo** a número.

Mirá qué pasa si no convertís:

```
edad = input("¿Tu edad? ")    # la persona escribe 10
print(edad + 5)               # ERROR: no se puede sumar texto + número
```

5.2. Conversión de tipos

Para convertir entre tipos usás funciones con el nombre del tipo: `int()` (a entero), `float()` (a decimal) y `str()` (a texto):

```
edad_texto = input("¿Tu edad? ")    # esto es str, ej "10"
edad = int(edad_texto)              # ahora es int: 10
print(edad + 5)                     # 15

# Forma compacta, muy común: pedir y convertir en una sola línea
```

```
nivel = int(input("¿Nivel de tu Pokémon? "))
print(f"Cuando suba, será nivel {nivel + 1}")
```

Función	Convierte a	Ejemplo	Resultado
<code>int(x)</code>	entero	<code>int("25")</code>	25
<code>float(x)</code>	decimal	<code>float("6.5")</code>	6.5
<code>str(x)</code>	texto	<code>str(25)</code>	"25"

Un programa completo de ejemplo



```
print("=== Registro de Entrenador ===")
nombre = input("Tu nombre: ")
ciudad = input("Tu ciudad: ")
edad = int(input("Tu edad: "))

print(f"\nEntrenador {nombre}, de {ciudad}.")
print(f"En 5 años vas a tener {edad + 5} años.")
```

NOTA

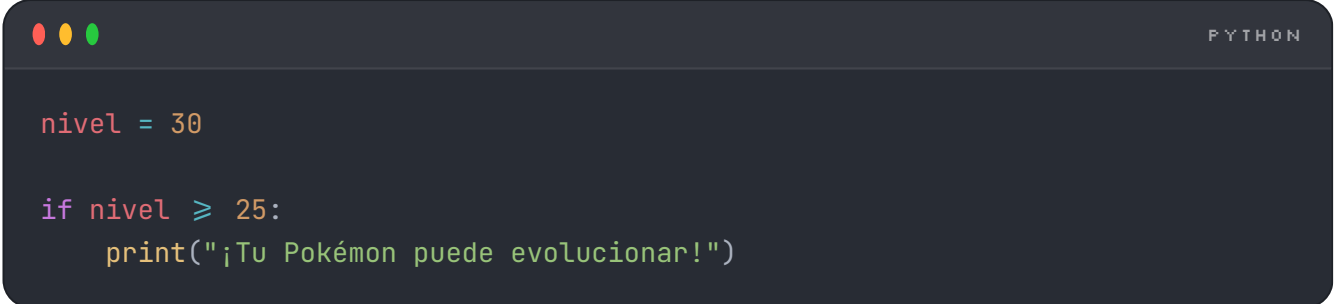
Si la persona escribe algo que no es un número (como "diez") y vos hacés `int(...)`, Python tira un `ValueError`. En el capítulo 11 vas a aprender a manejar ese error con `try/except`.

CAPÍTULO 6. Condicionales: if / elif / else

Hasta ahora tus programas corrían en línea recta. Con los **condicionales**, tu programa puede **tomar decisiones**: hacer una cosa u otra según lo que pase. Es como en una batalla: "si el rival es de Agua, uso un ataque Eléctrico".

6.1. if: hacer algo solo si...

if (si, en inglés) ejecuta un bloque de código **solo si** una condición es verdadera. La condición es una comparación que da **True** o **False**:



```

nivel = 30

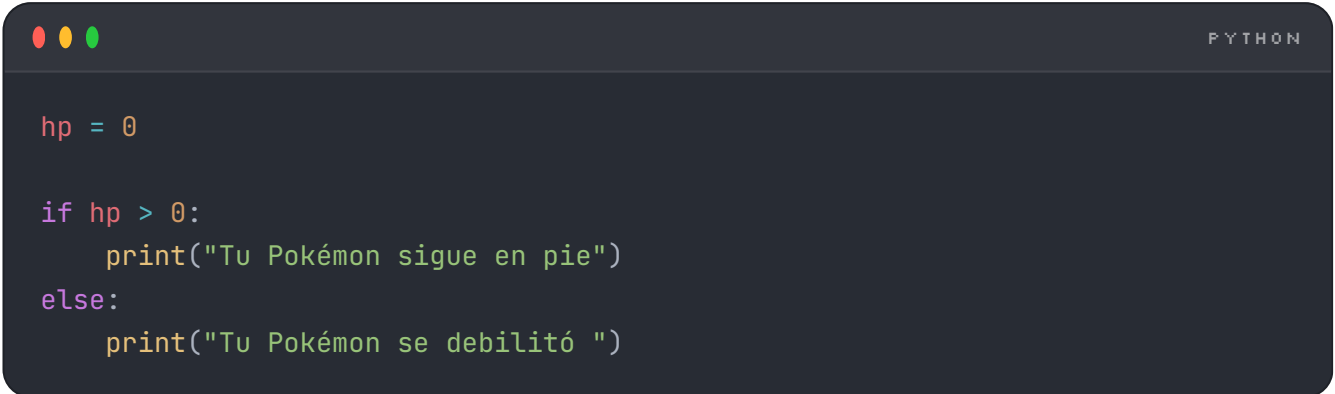
if nivel >= 25:
    print("¡Tu Pokémon puede evolucionar!")
  
```

CONCEPTO CLAVE

La indentación (sangría) importa y es obligatoria. El código "adentro" del **if** va con **4 espacios** de sangría. Python usa esa sangría para saber qué está adentro y qué afuera del **if**. No es decoración: si te equivocas con la sangría, da error.

6.2. else: el 'si no'

else (si no) define qué hacer cuando la condición del **if** **no** se cumple:



```

hp = 0

if hp > 0:
    print("Tu Pokémon sigue en pie")
else:
    print("Tu Pokémon se debilitó ")
  
```

6.3. elif: más caminos

Con `elif` (else if, "si no, si...") agregás más condiciones. Python las revisa **de arriba hacia abajo** y entra en la **primera** que sea verdadera; el resto las ignora:

```
hp = 40

if hp > 70:
    print("Tu Pokémon está sano ")
elif hp > 30:
    print("Tu Pokémon está cansado ")
elif hp > 0:
    print("¡Tu Pokémon está grave! ")
else:
    print("Tu Pokémon se debilitó ")
```

Como `hp` vale 40, no entra en el primer `if` (40 no es mayor a 70), pero sí en el primer `elif` (40 es mayor a 30), así que imprime "cansado" y se saltea los demás.

6.4. Combinar condiciones

Podés usar `and`, `or` y `not` (capítulo 3) dentro de un `if` para decisiones más ricas:

```
nivel = 30
tipo = "Fuego"

if nivel ≥ 25 and tipo == "Fuego":
    print("¡Charmeleon evoluciona a Charizard! ")

edad = 8
if edad < 10 or edad > 60:
    print("Precio especial")
```

Condicionales anidados

Podés poner un `if` adentro de otro (anidado). Cada nivel suma 4 espacios más de sangría:



PYTHON

```
tiene_medalla = True
nivel = 50

if tiene_medalla:
    if nivel ≥ 40:
        print("Podés entrar a la Liga Pokémon")
    else:
        print("Tenés la medalla, pero te falta nivel")
else:
    print("Primero conseguí una medalla")
```

CUIDADO

Errores típicos: (1) usar = en vez de == en la condición; (2) olvidar los dos puntos : al final del if; (3) mezclar espacios y tabs en la sangría. Si algo no anda, revisá esas tres cosas.

CAPÍTULO 7. Bucles: repetir cosas (while y for)

Un **bucle** (o ciclo) repite un bloque de código varias veces. Sin bucles, para atacar 5 veces tendrías que escribir `print()` cinco veces. Con bucles, lo escribís una vez y le decís cuántas veces repetir.

7.1. while: repetir mientras...

`while` (mientras) repite un bloque **mientras** una condición sea verdadera. Antes de cada vuelta revisa la condición; cuando se vuelve falsa, el bucle termina:

```

hp_rival = 100

while hp_rival > 0:
    print(f"Atacás. HP del rival: {hp_rival}")
    hp_rival = hp_rival - 20    # ¡importante! algo tiene que cambiar

print("¡Ganaste la batalla! ")

```

Eso imprime el HP bajando de 100 a 80, 60, 40, 20, y cuando llega a 0 corta y felicita.

CUIDADO

¡Cuidado con los bucles infinitos! Si la condición nunca se vuelve falsa, el programa no termina nunca (se "cuelga"). Asegurate de que algo cambie adentro del `while` que, tarde o temprano, haga falsa la condición. Si quedás en un bucle infinito, cortalo con **Ctrl + C**.

Acumuladores y contadores

Un patrón muy común: una variable que arranca en un valor y se va modificando en cada vuelta. Un contador cuenta; un acumulador suma:

```

total = 0          # acumulador
numero = 1
while numero ≤ 5:
    total = total + numero    # le voy sumando
    numero = numero + 1
print(total)           # 15  (1+2+3+4+5)

```

7.2. for: recorrer una secuencia

for (para cada) recorre los elementos de una secuencia, uno por uno. Es ideal cuando sabés sobre qué querés repetir (una lista, un texto, un rango de números):

```

equipo = ["Pikachu", "Charizard", "Snorlax"]
for pokemon in equipo:
    print(f"Tengo a {pokemon}")
# Tengo a Pikachu
# Tengo a Charizard
# Tengo a Snorlax

# También se puede recorrer un texto, letra por letra:
for letra in "Pika":
    print(letra)      # P, i, k, a (cada una en su línea)

```

range(): generar números

range() genera una secuencia de números, perfecta para repetir N veces. Ojo: el número final **no** se incluye:

```

for i in range(5):          # 0, 1, 2, 3, 4
    print(f"Pokéball número {i + 1}")

for nivel in range(1, 6):   # 1, 2, 3, 4, 5 (del 1 al 5)
    print(nivel)

for n in range(0, 11, 2):   # 0, 2, 4, 6, 8, 10 (de 2 en 2)
    print(n)

```

Forma	Genera
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(1, 6)</code>	1, 2, 3, 4, 5
<code>range(0, 11, 2)</code>	0, 2, 4, 6, 8, 10

7.3. break y continue

Dos instrucciones para controlar el bucle desde adentro: `break` **corta** el bucle por completo; `continue` **saltea** el resto de la vuelta actual y pasa a la siguiente:

```
equipo = ["Pikachu", "Mewtwo", "Snorlax"]

# break: dejamos de buscar apenas encontramos a Mewtwo
for pokemon in equipo:
    if pokemon == "Mewtwo":
        print("¡Encontramos a Mewtwo!")
        break
    print(f"{pokemon} no es Mewtwo, sigo buscando...")
```

```
# continue: salteamos a los Pokémon debilitados
equipo_hp = [100, 0, 50]
for hp in equipo_hp:
    if hp == 0:
        continue          # saltea este y va al siguiente
    print(f"Este Pokémon tiene {hp} HP y puede pelear")
```

7.4. Bucles anidados

Podés poner un bucle adentro de otro. Por cada vuelta del de afuera, el de adentro hace todas sus vueltas. Útil para combinaciones, tablas, grillas:

```
for entrenador in ["Ash", "Misty"]:
    for pokemon in ["Pikachu", "Staryu"]:
        print(f"{entrenador} podría usar a {pokemon}")
```

CAPÍTULO 8. Funciones

Una **función** es un bloque de código con nombre que escribís **una vez** y podés usar **muchas veces**. Sirve para no repetirte y para organizar tu programa en piezas con sentido.

ANALOGÍA POKÉMON

Es como un ataque que tu Pokémon **aprende una vez** y después usa mil veces. No reaprende "Impactrueno" en cada batalla: lo sabe, le pone un nombre, y lo invoca cuando quiere.

8.1. Definir y llamar una función

Se define con `def` (de define), un nombre, paréntesis y dos puntos. El cuerpo va indentado. **Definir** no la ejecuta; para ejecutarla hay que **llamarla** por su nombre con paréntesis:

```
def saludar():                # definición
    print("¡Hola, Entrenador!")

saludar()                    # llamada → imprime el saludo
saludar()                    # la usamos de nuevo, gratis
```

8.2. Parámetros: pasarle datos

Un **parámetro** es un dato que la función recibe para trabajar. Va entre los paréntesis. Cuando llamás la función, le pasás el valor (el **argumento**):

```
def saludar(nombre):
    print(f"¡Hola, {nombre}!")

saludar("Ash")                # ¡Hola, Ash!
saludar("Misty")              # ¡Hola, Misty!

# Varios parámetros, separados por comas:
def presentar(nombre, tipo):
    print(f"{nombre} es de tipo {tipo}")
```

```
presentar("Pikachu", "Eléctrico")
```

8.3. return: devolver un resultado

`return` hace que la función **devuelva** un valor para usarlo después. Es la diferencia entre una función que solo "muestra" y una que "calcula y entrega":

```
def calcular_dano(ataque, defensa):
    return ataque - defensa

resultado = calcular_dano(50, 20) # guardamos lo que devuelve
print(resultado)                 # 30
print(calcular_dano(80, 30))     # 50 (también se puede usar directo)
```

CONCEPTO CLAVE

print vs return: print solo MUESTRA algo en pantalla; return DEVUELVE un valor que tu programa puede seguir usando (guardar, sumar, comparar...). Si querés usar el resultado después, usá return. Apenas Python ejecuta un return, sale de la función.

Devolver varios valores

Una función puede devolver más de un valor (en realidad devuelve una tupla, capítulo 9):

```
def stats(nivel):
    hp = nivel * 3
    ataque = nivel * 2
    return hp, ataque

vida, ataque = stats(10) # 30, 20
print(vida, ataque)      # 30 20
```

8.4. Valores por defecto

Podés darle a un parámetro un valor por defecto, que se usa si al llamar no pasás ese dato:



PYTHON

```
def atacar(nombre, dano=10):      # dano vale 10 si no se especifica
    print(f"{nombre} hizo {dano} de daño")

atacar("Pikachu")                # Pikachu hizo 10 de daño (usa el default)
atacar("Charizard", 35)          # Charizard hizo 35 de daño (lo pisamos)
```

8.5. Scope: dónde vive cada variable

El **scope** (alcance) es la zona donde una variable existe. Una variable creada **adentro** de una función solo existe ahí adentro (es local). Afuera no se la ve:



PYTHON

```
def entrenar():
    secreto = "técnica oculta"    # variable LOCAL
    print(secreto)

entrenar()
# print(secreto)    # ERROR: 'secreto' no existe afuera de la función
```

8.6. Funciones lambda

Una **lambda** es una función chiquita de una sola línea, sin nombre. Útil para cosas simples. Su forma es

`lambda parámetros: expresión`:



PYTHON

```
doble = lambda x: x * 2
print(doble(5))    # 10

sumar = lambda a, b: a + b
print(sumar(3, 4))    # 7
```

8.7. Recursión

La **recursión** es cuando una función se llama a sí misma para resolver un problema más chico. Siempre necesita un **caso base**: una condición que la frene (si no, se llama infinitamente y revienta):



PYTHON

```
def factorial(n):  
    if n ≤ 1:                # caso base: frena acá  
        return 1  
    return n * factorial(n - 1)  # caso recursivo: se llama con n-1  
  
print(factorial(5))    # 120  (5*4*3*2*1)
```

Es como muñecas rusas: cada `factorial` abre otra más chica, hasta llegar a la más chiquita (el caso base), y ahí se arma el resultado para arriba.

CAPÍTULO 9. Listas y tuplas

Hasta ahora cada variable guardaba **un** dato. Una **lista** guarda **muchos** datos juntos, en orden, bajo un solo nombre. Es como tu equipo Pokémon. Se escribe entre corchetes `[ ]`, con los elementos separados por comas:

```

equipo = ["Pikachu", "Charizard", "Snorlax"]
niveles = [25, 50, 40]
vacía = [] # una lista vacía
mezcla = ["Pikachu", 25, True] # puede tener tipos distintos

```

9.1. Acceder y modificar

Cada elemento tiene una **posición (índice)**, que empieza en 0:

```

equipo = ["Pikachu", "Charizard", "Snorlax"]
print(equipo[0]) # Pikachu (el primero)
print(equipo[1]) # Charizard
print(equipo[-1]) # Snorlax (el último)

equipo[0] = "Raichu" # cambiar un elemento (Pikachu evolucionó)
print(equipo) # ['Raichu', 'Charizard', 'Snorlax']

print(len(equipo)) # 3 (cuántos hay)
print("Snorlax" in equipo) # True (¿está?)

```

9.2. Métodos de listas

Las listas tienen métodos para agregar, quitar, ordenar, etc.:

```

equipo = ["Pikachu", "Charizard"]

equipo.append("Snorlax") # agrega al final
equipo.insert(0, "Mewtwo") # inserta en la posición 0
equipo.remove("Charizard") # borra por valor

```



```
ultimo = equipo.pop()      # saca y devuelve el último
equipo.sort()              # ordena (alfabético o numérico)
equipo.reverse()           # da vuelta el orden
```

Método / función	Qué hace
<code>.append(x)</code>	Agrega x al final
<code>.insert(i, x)</code>	Inserta x en la posición i
<code>.remove(x)</code>	Borra la primera aparición de x
<code>.pop()</code>	Saca y devuelve el último
<code>.sort()</code> / <code>.reverse()</code>	Ordena / invierte
<code>len(lista)</code>	Cantidad de elementos
<code>x in lista</code>	¿x está en la lista?

Y por supuesto, recorrerás una lista con un `for` (capítulo 7):

```

niveles = [25, 50, 40]
total = 0
for n in niveles:
    total = total + n
print(f"Nivel total del equipo: {total}")  # 115

```

9.3. enumerate y zip

`enumerate` te da la posición **y** el valor a la vez. `zip` recorre dos listas en paralelo:

```

equipo = ["Pikachu", "Charizard"]
for i, pokemon in enumerate(equipo, start=1):
    print(f"{i}. {pokemon}")
# 1. Pikachu
# 2. Charizard

```

```
nombres = ["Pikachu", "Charizard"]
niveles = [25, 50]
for nombre, nivel in zip(nombres, niveles):
    print(f"{nombre} es nivel {nivel}")
```

9.4. Comprensiones de listas

Una **comprensión** crea una lista nueva a partir de otra, en una sola línea. Es muy "pythónico" y, una vez que te acostumbras, súper cómodo:

```
niveles = [25, 50, 40, 15]

# Forma larga (con for):
dobles = []
for n in niveles:
    dobles.append(n * 2)

# La misma cosa, en una línea (comprensión):
dobles = [n * 2 for n in niveles]      # [50, 100, 80, 30]

# Con condición (filtrar): solo los niveles altos
altos = [n for n in niveles if n ≥ 40] # [50, 40]
```

La estructura es: `[expresión for elemento in coleccion if condición]`.

9.5. Tuplas

Una **tupla** es como una lista, pero **inmutable**: una vez creada, no se puede cambiar. Se escribe con paréntesis `()`. Se usa para datos que no deberían cambiar (como una coordenada):

```
coordenada = (10, 20)
print(coordenada[0])      # 10
# coordenada[0] = 5      # ERROR: una tupla no se puede modificar

# Desempaquetado: asignar varios a la vez
x, y = coordenada
print(x, y)               # 10 20
```

NOTA

¿Lista o tupla? Usá **lista** cuando los datos van a cambiar (tu equipo, que evoluciona). Usá **tupla** cuando son fijos (las coordenadas de un punto, los días de la semana).

CAPÍTULO 10. Diccionarios y conjuntos

10.1. Diccionarios: pares clave → valor

Un **diccionario** guarda pares de **clave: valor**. En vez de acceder por posición (como las listas), accedés por una **clave** con sentido. Es como una Pokédex de verdad: buscás por nombre y obtenés los datos. Se escribe con llaves `{ }`:

```
pikachu = {
    "nombre": "Pikachu",
    "tipo": "Electrico",
    "nivel": 25,
    "hp": 100,
}

print(pikachu["nombre"])    # Pikachu (accedo por la clave)
print(pikachu["nivel"])    # 25
```

Modificar, agregar y acceso seguro

```
pikachu["nivel"] = 26        # cambiar un valor existente
pikachu["ataque"] = 55       # agregar una clave nueva

# .get() devuelve un valor por defecto si la clave NO existe (no rompe):
print(pikachu.get("defensa", 0))  # 0 (no existe "defensa", devuelve 0)
print("nivel" in pikachu)        # True (¿existe esa clave?)
```

Recorrer un diccionario

Con `.items()` obtenés la clave y el valor de cada par:

```
for clave, valor in pikachu.items():
    print(f"{clave}: {valor}")
# nombre: Pikachu
# tipo: Electrico
```

```
# nivel: 26
# ...
```

Los diccionarios se pueden anidar (un dict adentro de otro) para datos más complejos. Por ejemplo, una Pokédex es un dict de Pokémon, donde cada Pokémon es a su vez un dict:

```
pokedex = {
    "pikachu": {"tipo": "Electrico", "nivel": 25},
    "charizard": {"tipo": "Fuego", "nivel": 50},
}
print(pokedex["charizard"]["tipo"]) # Fuego
```

10.2. Conjuntos (sets)

Un **set** (conjunto) guarda elementos **únicos**: sin repetidos y sin un orden definido. Se escribe con llaves `{ }` (pero sin pares clave:valor). Es ideal para sacar duplicados:

```
tipos = {"Fuego", "Agua", "Fuego", "Planta"}
print(tipos) # {'Fuego', 'Agua', 'Planta'} (el Fuego repetido
desaparece)

capturados = ["Pikachu", "Pidgey", "Pikachu", "Rattata"]
unicos = set(capturados)
print(len(unicos)) # 3 (sin contar el Pikachu repetido)

tipos.add("Electrico") # agregar
print("Agua" in tipos) # True
```

10.3. Resumen de colecciones

Ya viste las cuatro colecciones principales de Python. Cuándo usar cada una:

Colección	Símbolo	Característica	Cuándo usarla
Lista	<code>[]</code>	Ordenada, modificable	Una serie de cosas que cambia
Tupla	<code>()</code>	Ordenada, inmutable	Datos fijos que no cambian

Colección	Símbolo	Característica	Cuándo usarla
Set	<code>{ }</code>	Única, sin orden	Elementos sin repetir
Diccionario	<code>{c: v}</code>	Pares clave-valor	Buscar datos por nombre/clave

CAPÍTULO 11. Archivos y manejo de errores

Hasta ahora, cuando cerrabas tu programa, todos los datos se perdían. Para que **sobrevivan**, los guardás en **archivos** del disco. Así tu programa tiene "memoria" permanente.

11.1. Abrir archivos con with

La forma recomendada de trabajar con archivos es `with open(...)`, que se encarga de cerrar el archivo solo, aunque haya un error. `open()` recibe el nombre y el **modo**:

Modo	Significa
"r"	leer (read) — el archivo debe existir
"w"	escribir (write) — crea o REEMPLAZA todo
"a"	agregar (append) — escribe al final

```

# Escribir (¡el modo "w" pisa lo que había!)
with open("equipo.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Pikachu\n")      # \n = salto de línea
    archivo.write("Charizard\n")

# Al salir del 'with', el archivo se cierra solo.

# Leer todo el contenido de una:
with open("equipo.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()
    print(contenido)

# Leer línea por línea (lo más común):
with open("equipo.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea.strip())      # strip() saca el \n del final

```

NOTA

El `encoding="utf-8"` evita problemas con tildes y la ñ. Acostumbrate a ponerlo siempre que abras un archivo de texto.

11.2. CSV: datos en filas y columnas

Un **CSV** (valores separados por comas) es un archivo donde cada línea es una fila y los datos van separados por comas. Es como una mini planilla. Lo podés manejar con `split`/`join` (capítulo 4) o con el módulo `csv`:

```
import csv

# Escribir un CSV:
with open("pokedex.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.writer(f)
    escritor.writerow(["nombre", "tipo", "nivel"]) # encabezado
    escritor.writerow(["Pikachu", "Electrico", 25])
    escritor.writerow(["Charizard", "Fuego", 50])

# Leer un CSV:
with open("pokedex.csv", "r", encoding="utf-8") as f:
    lector = csv.reader(f)
    for fila in lector:
        print(fila) # cada fila es una lista: ['Pikachu', 'Electrico', '25']
```

11.3. try / except: manejar errores

Algunas cosas pueden fallar: abrir un archivo que no existe, convertir "abc" a número, dividir por cero. Sin protección, el programa se cae. Con `try`/`except` **atrapás** el error y seguís:

```
try:
    # Código que PODRÍA fallar
    numero = int(input("Nivel: "))
    print(f"El nivel es {numero}")
except ValueError:
    # Esto corre SOLO si hubo un ValueError
    print("Eso no es un número válido")
```

Atrapar errores al abrir archivos



PYTHON


```
try:
    with open("noexiste.txt", "r") as f:
        contenido = f.read()
except FileNotFoundError:
    print("El archivo no existe, arranco vacío")
    contenido = ""
```

Error común	Cuándo pasa
<code>ValueError</code>	Conversión inválida (<code>int("abc")</code>)
<code>FileNotFoundError</code>	Abrir un archivo que no existe
<code>KeyError</code>	Pedir una clave que no está en un diccionario
<code>ZeroDivisionError</code>	Dividir por cero
<code>IndexError</code>	Pedir una posición que no existe en una lista

TIP

Atrapa errores **específicos** (como `ValueError`), no un `except:` pelado que atrapa todo. Así sabés exactamente qué estás manejando y no te ocultás bugs sin querer.

CAPÍTULO 12. Programación orientada a objetos (POO)

La **POO** es una forma de organizar tu código creando tus propios "moldes". Una **clase** es el molde (la especie "Pikachu", el plano). Un **objeto** (o instancia) es un individuo hecho con ese molde (tu Pikachu concreto, distinto del Pikachu de Ash).

ANALOGÍA POKÉMON

Mismo molde, datos distintos: la clase Pokemon define que todo Pokémon tiene nombre, nivel y HP, y que puede atacar. Pero tu Pikachu y el Pikachu de Ash son objetos distintos, con sus propios valores.

12.1. Definir una clase

Se define con `class` y, por convención, el nombre empieza con **mayúscula**. El método especial `__init__` es el **constructor**: se ejecuta automáticamente al crear el objeto y le da sus datos iniciales. `self` es "el propio objeto":

```
class Pokemon:
    def __init__(self, nombre, tipo, nivel):
        # 'self' es este objeto. Guardamos sus datos (atributos).
        self.nombre = nombre
        self.tipo = tipo
        self.nivel = nivel
        self.hp = 100

# Crear objetos (instancias) con el molde:
pikachu = Pokemon("Pikachu", "Electrico", 25)
charizard = Pokemon("Charizard", "Fuego", 50)

print(pikachu.nombre)    # Pikachu    (accedo a un atributo con punto)
print(charizard.nivel)   # 50
```

CONCEPTO CLAVE

`self` siempre es el **primer parámetro** de los métodos, pero **no lo pasás** al llamarlos: Python lo manda solo. `self.nombre = nombre` significa "guardá este nombre adentro mío".

12.2. Métodos: las acciones del objeto

Un **método** es una función definida dentro de la clase. Son las acciones que el objeto puede hacer. Siempre reciben `self` primero, y así pueden usar y modificar los atributos del objeto:

```
class Pokemon:
    def __init__(self, nombre, tipo):
        self.nombre = nombre
        self.tipo = tipo
        self.hp = 100

    def atacar(self):
        return f"{self.nombre} ataca con un golpe de tipo {self.tipo}!"

    def recibir_dano(self, cantidad):
        self.hp = self.hp - cantidad
        if self.hp < 0:
            self.hp = 0

pikachu = Pokemon("Pikachu", "Electrico")
print(pikachu.atacar())      # Pikachu ataca con un golpe de tipo Electrico!
pikachu.recibir_dano(30)
print(pikachu.hp)           # 70
```

12.3. `__str__`: cómo se muestra el objeto

Si hacés `print(pikachu)` sin más, Python muestra algo feo como `<__main__.Pokemon object at 0x...>`. El método especial `__str__` define un texto lindo:

```
class Pokemon:
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel

    def __str__(self):
        return f"{self.nombre} (Nivel {self.nivel})"

pikachu = Pokemon("Pikachu", 25)
print(pikachu)      # Pikachu (Nivel 25) ← ¡mucho mejor!
```

NOTA

También existe `__repr__`, parecido pero pensado para el programador (lo ves en el REPL y cuando un objeto está dentro de una lista). Regla: `__str__` para el usuario (lindo), `__repr__` para vos (preciso).

12.4. Encapsulamiento básico

Encapsular es proteger los datos internos. Por convención, un atributo que no debería tocarse desde afuera se nombra con un **guión bajo** adelante (`_hp`), y se cambia solo a través de métodos que pueden validar:

```
class Pokemon:
    def __init__(self, nombre):
        self.nombre = nombre
        self._hp = 100      # "interno": no lo toques directo

    def curar(self, cantidad):
        self._hp = self._hp + cantidad
        if self._hp > 100:   # el método valida que no pase de 100
            self._hp = 100
```

CAPÍTULO 13. POO avanzado: herencia y polimorfismo

13.1. Herencia

La **herencia** deja que una clase "hija" reciba todo lo de una clase "padre", y le agregue o cambie lo suyo. Así escribís lo común una vez. La hija se define poniendo el padre entre paréntesis:

```
class Pokemon:                                # clase PADRE
    def __init__(self, nombre, nivel):
        self.nombre = nombre
        self.nivel = nivel
    def atacar(self):
        return f"{self.nombre} ataca!"

class PokemonFuego(Pokemon):                  # HIJA: hereda de Pokemon
    def lanzallamas(self):
        return f"{self.nombre} usa Lanzallamas! "

charizard = PokemonFuego("Charizard", 50)
print(charizard.atacar())                     # heredado de Pokemon
print(charizard.lanzallamas())                # propio de PokemonFuego
```

super(): llamar al padre

Cuando la hija define su propio `__init__`, usa `super()` para no repetir el del padre:

```
class PokemonFuego(Pokemon):
    def __init__(self, nombre, nivel):
        super().__init__(nombre, nivel)    # ejecuta el __init__ del padre
        self.tipo = "Fuego"                # y agrega lo propio
```

13.2. Polimorfismo

Polimorfismo ("muchas formas") significa que el mismo método se comporta distinto según la clase. Todos los Pokémon tienen `atacar()`, pero cada tipo lo **sobrescribe** a su manera:



PYTHON

```

class Pokemon:
    def __init__(self, nombre):
        self.nombre = nombre
    def atacar(self):
        return f"{self.nombre} usa un ataque normal"

class PokemonFuego(Pokemon):
    def atacar(self):      # SOBREScribe el método del padre
        return f"{self.nombre} usa Lanzallamas! "

class PokemonAgua(Pokemon):
    def atacar(self):
        return f"{self.nombre} usa PistoLa Agua! "

equipo = [PokemonFuego("Charizard"), PokemonAgua("Blastoise")]
for p in equipo:
    print(p.atacar())    # cada uno responde a su manera

```

CONCEPTO CLAVE

El poder del polimorfismo: tratás a todos como "Pokémon" y cada uno hace lo suyo, sin tener que escribir `if tipo == "fuego" ... elif tipo == "agua" ...` por todos lados. El código queda mucho más limpio.

13.3. property, métodos de clase y abstractas

Hay más herramientas avanzadas que vas a ver en la semana 9 del curso:

- `@property` — deja usar un método como si fuera un atributo (sin paréntesis), ideal para calcular o validar valores.
- `@staticmethod` y `@classmethod` — funciones dentro de la clase con usos especiales.
- Clases abstractas (módulo `abc`) — un "contrato" que obliga a las hijas a implementar ciertos métodos.

```

class Pokemon:
    def __init__(self, hp):
        self._hp = hp

    @property
    def hp(self):      # se accede como pokemon.hp (sin paréntesis)
        return self._hp

```

```
@hp.setter
def hp(self, valor):    # se ejecuta al hacer pokemon.hp = algo
    self._hp = max(0, valor)  # nunca negativo

p = Pokemon(100)
p.hp = -50           # el setter lo corrige
print(p.hp)          # 0
```

CAPÍTULO 14. Módulos, pip y entornos virtuales

Un **módulo** es un archivo lleno de código útil que podés **importar** y usar. Python trae muchísimos "de fábrica" (la librería estándar). No reinventes la rueda: aprovechá el trabajo de millones de personas.

14.1. import y la librería estándar

Para usar un módulo, lo importás. Después accedés a sus funciones con un punto:

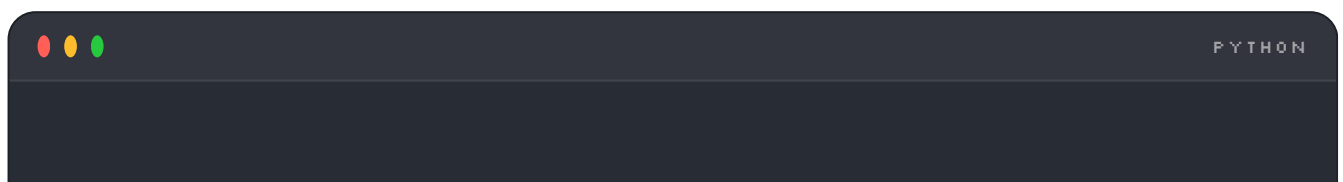
```
import math
import random
from datetime import date

print(math.sqrt(16))           # 4.0    (raíz cuadrada)
print(math.ceil(4.2))         # 5     (redondea para arriba)
print(random.randint(1, 6))    # un número al azar entre 1 y 6 (dado)
print(random.choice(["Pikachu", "Onix"])) # uno al azar de la lista
print(date.today())           # la fecha de hoy
```

Módulo	Sirve para
<code>math</code>	Matemática (raíz, redondeo, pi...)
<code>random</code>	Azar (números al azar, elegir, mezclar)
<code>datetime</code>	Fechas y horas
<code>os</code> / <code>sys</code>	Interactuar con el sistema
<code>json</code>	Guardar/cargar datos estructurados

14.2. json: guardar datos estructurados

JSON es el formato para guardar datos (diccionarios, listas) en un archivo. Lo vas a usar en los proyectos finales:




```
import json

datos = {"nombre": "Pikachu", "nivel": 25}

# Guardar en un archivo:
with open("pokemon.json", "w") as f:
    json.dump(datos, f)

# Cargar de un archivo:
with open("pokemon.json", "r") as f:
    datos = json.load(f)
print(datos["nombre"])    # Pikachu
```

14.3. Tus propios módulos

Cualquier archivo `.py` tuyo es un módulo que podés importar desde otro. Así organizás proyectos grandes. Si tenés `pokeutils.py` con una función `formatear_nombre`, desde otro archivo hacés:

```
import pokeutils
print(pokeutils.formatear_nombre("pikachu"))    # Pikachu
```

14.4. pip y entornos virtuales

La librería estándar es enorme, pero a veces necesitás algo externo. **pip** es el instalador de paquetes de Python (tu PokéMart de librerías):

```
pip install requests          # instala una librería
pip install -r requirements.txt # instala todas las de una lista
```

NOTA

Un **entorno virtual** (venv) es una cajita aislada con las librerías de tu proyecto, separada del Python del sistema. Así cada proyecto tiene sus versiones sin pisarse. El `setup.sh` del curso lo crea por vos.

14.5. requests: traer datos de internet

Con la librería `requests` tu programa puede pedir datos a un servicio de internet (una **API**). La **PokéAPI** tiene datos de todos los Pokémon, gratis:

```
import requests

respuesta = requests.get("https://pokeapi.co/api/v2/pokemon/pikachu")
datos = respuesta.json()      # convierte la respuesta a un diccionario

print(datos["name"])          # pikachu
print(datos["height"])        # altura
print(datos["types"][0]["type"]["name"]) # electric
```

NOTA

Para usar `requests` necesitas internet y tenerla instalada (`pip install requests`). En la semana 10 del curso construís una Pokédex online que muestra datos reales con esto.

CAPÍTULO 15. Linux: la terminal

Esta parte cubre **Linux**, que en el curso ves en las semanas 1 y 2 (antes de Python). La **terminal** es una ventana donde escribís comandos de texto y la computadora te responde. Al principio asusta, pero es la herramienta más poderosa que vas a tener.

ANALOGÍA POKÉMON

Pensá en la terminal como la **Pokédex de tu sistema**: una herramienta de texto donde cada comando es un "ataque". Aprender los comandos es como aprender los movimientos de tu Pokémon.

Cuando abrís la terminal, ves un prompt tipo `felipe@maquina:~$`. El `~` significa "tu carpeta personal" (tu home) y el `$` dice "estoy listo, escribí tu comando".

15.1. Moverte por las carpetas

Todo en Linux está organizado en **carpetas** (directorios), una dentro de otra, formando un árbol. Estos tres comandos te dejan ubicarte y viajar:

```

B A S H

pwd          # ¿dónde estoy? muestra la ruta de la carpeta actual
ls           # ver qué hay acá (archivos y carpetas)
ls -l        # en formato largo: permisos, tamaño, fecha
ls -a        # incluye archivos ocultos (empiezan con un punto)
cd pokecenter # entrar a la carpeta pokecenter
cd ..        # subir un nivel (a la carpeta de arriba)
cd ~         # ir a tu carpeta personal (home)
cd /         # ir a la raíz de todo

```

Las opciones como `-l` o `-a` se llaman **flags**: modifican lo que hace el comando. Es como un objeto que potencia el ataque.

15.2. Crear, copiar y borrar

```

B A S H

mkdir pokecenter # crear una carpeta (make directory)
mkdir -p a/b/c   # crear carpetas anidadas de una (-p)
touch pikachu.txt # crear un archivo vacío

```

```
echo "Electrico" > pikachu.txt # escribir texto DENTRO de un archivo
cat pikachu.txt               # mostrar el contenido de un archivo
cp pikachu.txt copia.txt      # copiar un archivo
cp -r carpeta copia           # copiar una carpeta entera (-r = recursivo)
mv pikachu.txt raichu.txt     # renombrar (o mover a otra carpeta)
rm pikachu.txt                # borrar un archivo
rm -r carpeta                 # borrar una carpeta y todo lo de adentro
```

CUIDADO

En Linux **no hay papelera de reciclaje**: lo que borras con `rm`, se va para siempre. Revisá siempre antes de borrar. Y **nunca, jamás** corras `rm -rf /`: intenta borrar todo el sistema.

15.3. Rutas absolutas y relativas

Una **ruta** es la dirección de un archivo o carpeta. Una ruta **absoluta** empieza en la raíz `/` y vale desde cualquier lado (como una dirección completa). Una **relativa** parte de donde estás parado:

```
cd /home/ash/pokecenter # ruta ABSOLUTA (empieza con /)
cd pokecenter/gimnasio  # ruta RELATIVA (desde donde estás)
```

- `.` → la carpeta actual (acá mismo)
- `..` → la carpeta de arriba
- `~` → tu home
- `/` → la raíz de todo

15.4. Permisos básicos

Cada archivo tiene **permisos** que dicen quién puede leerlo, escribirlo o ejecutarlo. Al correr `ls -l` ves algo como `-rwxr-xr--`. Esa columna se lee así:

- `r` = leer, `w` = escribir, `x` = ejecutar.
- Se agrupan de a 3, para tres grupos: el **dueño**, el **grupo** y los **otros**.

En la próxima parte vas a aprender a cambiar estos permisos con `chmod`.

15.5. Resumen

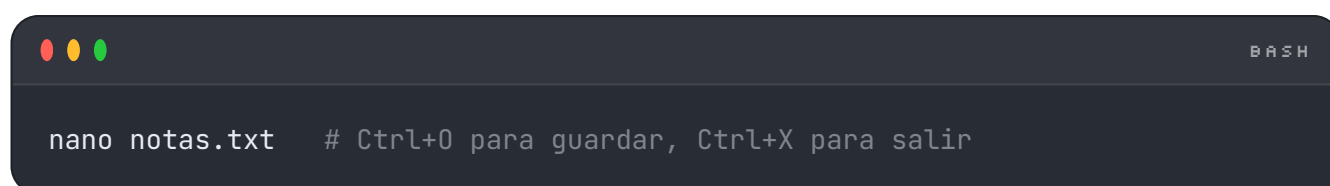
Comando	Qué hace
<code>pwd</code>	Dice en qué carpeta estás
<code>ls</code>	Lista archivos y carpetas
<code>cd</code>	Cambia de carpeta
<code>mkdir</code>	Crea una carpeta
<code>touch</code> / <code>echo</code>	Crea / escribe archivos
<code>cat</code>	Muestra un archivo
<code>cp</code> / <code>mv</code>	Copia / mueve o renombra
<code>rm</code>	Borra (¡sin papelera!)

CAPÍTULO 16. Linux: comandos intermedios

Ahora que te movés por la terminal, sumamos superpoderes: editar archivos, encadenar comandos, buscar, manejar procesos y automatizar con scripts.

16.1. Editar con nano

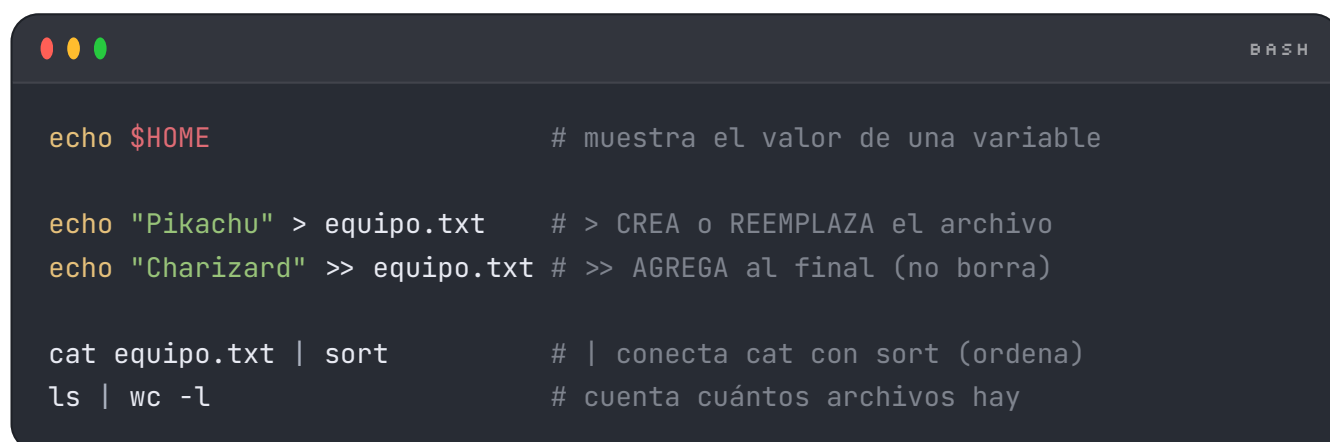
`nano` es un editor de texto simple que vive dentro de la terminal. Abajo te muestra los atajos (el  significa la tecla Ctrl):



```
nano notas.txt # Ctrl+O para guardar, Ctrl+X para salir
```

16.2. Variables, redirección y pipes

Una **variable de entorno** guarda un dato con un nombre. La **redirección** manda la salida de un comando a un archivo, y un **pipe** () conecta la salida de un comando con otro:



```
echo $HOME # muestra el valor de una variable

echo "Pikachu" > equipo.txt # > CREA o REEMPLAZA el archivo
echo "Charizard" >> equipo.txt # >> AGREGA al final (no borra)

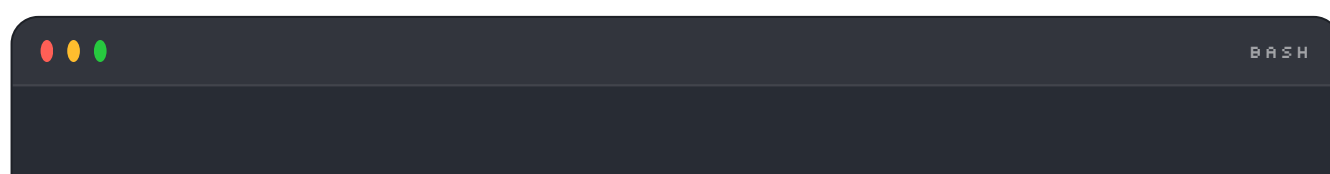
cat equipo.txt | sort # | conecta cat con sort (ordena)
ls | wc -l # cuenta cuántos archivos hay
```

CUIDADO

Ojo: > pisa todo lo que había en el archivo; >> agrega al final. Confundirlos borra datos.

16.3. Buscar: grep y find

`grep` busca **texto dentro** de archivos. `find` busca **los archivos en sí** por su nombre:



```
find
```

```
grep "Fuego" pokedex.txt      # muestra las líneas que contienen "Fuego"
grep -i "fuego" pokedex.txt  # -i ignora mayúsculas/minúsculas
find . -name "*.txt"         # busca todos los .txt desde la carpeta actual
```

16.4. Procesos y permisos

Un **proceso** es un programa corriendo. `chmod` cambia los permisos de un archivo (lo más común: dar permiso de ejecución con `+x`):

```
ps aux          # lista todos los procesos en ejecución
ps aux | grep python # filtra los de python
kill 1234       # cierra el proceso con ese número (PID)
chmod +x script.sh # da permiso de EJECUCIÓN al archivo
chmod 755 script.sh # forma numérica (rwx para el dueño, r-x para el resto)
```

16.5. Tu primer script bash

Un **script** es un archivo de texto con una lista de comandos que se ejecutan en orden. Es tu máquina automática. La primera línea (el "shebang") dice que es un script bash:

```
#!/usr/bin/env bash
# Este script saluda y registra un Pokémon capturado.
echo "¡Hola, Entrenador!"
echo "Capturaste a: $1"          # $1 es el primer argumento
echo "$1" >> capturados.txt      # lo guarda en un archivo
```

Para correrlo, le das permiso de ejecución y lo ejecutás con `./`:

```
chmod +x capturar.sh
./capturar.sh Snorlax # imprime y registra a Snorlax
```

16.6. Instalar programas y SSH

En Ubuntu/Debian, `apt` instala programas. `ssh` te deja controlar otra computadora por la terminal, de forma segura:

```

sudo apt update          # actualiza la lista de programas
sudo apt install cowsay  # instala un programa
ssh entrenador@192.168.1.50 # te conectás a otra máquina

```

NOTA

`sudo` ejecuta un comando como administrador (root). Es poderoso: no corras con `sudo` cosas que no entendés.

16.7. Resumen

Comando	Qué hace
<code>nano</code>	Edita un archivo de texto
<code>></code> / <code>>></code>	Redirige (reemplaza / agrega)
<code> </code>	Conecta la salida de un comando con otro
<code>grep</code> / <code>find</code>	Busca texto / archivos
<code>ps</code> / <code>kill</code>	Ver / cerrar procesos
<code>chmod</code>	Cambia permisos
<code>apt</code> / <code>ssh</code>	Instala programas / conecta a otra máquina

CAPÍTULO 17. Mapa de temas del curso

Estos son **todos los temas** del curso, semana por semana. Cada semana tiene su teoría, ejercicios, soluciones, tests y un programa interactivo.

17.1. Fase 1 — Linux

Semana	Temas
01 — Fundamentos	Terminal, navegación, <code>ls cd pwd mkdir rm cp mv cat echo</code> , rutas, permisos
02 — Intermedio	<code>nano</code> , variables, scripts bash, <code>> >> </code> , <code>grep find ps kill chmod apt</code> , SSH

17.2. Fase 2 — Python básico

Semana	Temas
03 — Introducción	Variables, tipos, <code>print</code> , <code>input</code> , f-strings
04 — Control de flujo	<code>if/elif/else</code> , comparadores, lógicos, <code>while/for/range</code> , <code>break/continue</code>
05 — Funciones	<code>def</code> , parámetros, <code>return</code> , scope, <code>lambda</code> , recursión
06 — Listas y colecciones	Listas, tuplas, sets, diccionarios, comprensiones, <code>enumerate</code> , <code>zip</code>
07 — Cadenas y archivos	Métodos de string, slicing, <code>open</code> , CSV, <code>try/except</code>

17.3. Descanso — Git

Git: Control de Versiones — `init`, `add`, `commit`, `log`, ramas (`branch/switch/merge`), GitHub, `push/pull/clone`, `.gitignore`. Pensada para hacerse después de la semana 7.

17.4. Fase 3 — Python avanzado

Semana	Temas
08 — POO introducción	Clases, <code>__init__</code> , atributos, métodos, <code>self</code> , <code>__str__</code> , <code>__repr__</code>
09 — POO avanzado	Herencia, <code>super()</code> , polimorfismo, <code>@property</code> , clases abstractas (<code>abc</code>)
10 — Módulos y pip	<code>import</code> , módulos estándar, módulos propios, <code>pip</code> , <code>venv</code> , <code>requests</code>

17.5. Fase 4 — Proyectos

- **Semana 11 — Agenda del Entrenador:** app de consola modular con persistencia en JSON.
- **Semana 12 — Pokédex Web:** web con Flask + PokéAPI.
- **proyectos/pokedex-cli:** Pokédex de consola con sprites ASCII y favoritos.
- **proyectos/batalla-pokemon:** simulador con tipos, PP y estados alterados.
- **proyectos/agenda-entrenador** y **proyectos/pokedex-web:** versiones pulidas.

CAPÍTULO 18. Tests, preguntas frecuentes y glosario

18.1. Cómo correr los tests

Los **tests** te dicen si tus ejercicios están bien. Usamos **pytest**:

```
pytest # todos los tests
pytest semana-04-python-control-de-flujo/ # una semana
pytest semana-05-python-funciones/test_ejercicios.py -v # con detalle
```

NOTA

Verde = aprobado. Rojo = revisá tu solución. Por defecto los tests prueban las soluciones; la Liga los corre contra TU `ejercicios.py` para darte EXP y decirte qué ejercicio falta.

18.2. Preguntas frecuentes

¿Por dónde empiezo? Por la Liga: `python aventura.py`. Después seguí la semana 01.

Me sale "command not found: python". Probá con `python3`.

Los tests me dan rojo. ¿Está mal? Significa que tu solución todavía no es correcta. Leé el error (está en español), corregí y volvé a probar. Es parte normal del aprendizaje.

¿Puedo ver las soluciones? Sí, están en `soluciones.py`. Pero intentá primero vos: mirar sin intentar es como usar un truco, ganás pero no aprendés.

¿Necesito internet? Solo para algunas partes (la semana 10, la Pokédex online y el autocompletado web). El resto funciona sin conexión.

18.3. Glosario

Palabra	Qué significa
Terminal / consola	La ventana donde escribís comandos de texto.
Variable	Una "caja con nombre" donde guardás un dato.
Tipo	La clase de dato: int, float, str, bool...

Palabra	Qué significa
Función	Un bloque de código con nombre que podés reusar.
Parámetro / argumento	El dato que recibe / le pasás a una función.
Bucle	Una estructura que repite código (while, for).
Lista / diccionario	Colecciones que guardan muchos datos.
Clase / objeto	El molde / un individuo hecho con ese molde (POO).
Test	Un programa que revisa si tu código funciona bien.
EXP	Puntos de experiencia que ganás en la Liga al pasar tests.
Commit	Un punto de guardado de tu código en Git.
venv	Entorno virtual: cajita aislada con las librerías del curso.

"El mejor momento para empezar a programar fue ayer. El segundo mejor momento es ahora."
¡A atraparlos a todos!